

# Case Study: Financial Process Engineering & Automation

|                |                                         |
|----------------|-----------------------------------------|
| Client         | Nexoria Commerce Inc.                   |
| Specialization | FP&A, Financial Systems, and Automation |
| Author         | Portfolio Project                       |

## Executive Summary

Nexoria Commerce Inc. is a B2B and retail product distributor operating across 12 US states. Prior to this project, the company's 5-person finance team spent **60% of their operational hours** on manual, repetitive administrative processes, including invoice generation, multi-state tax calculation, shipping freight reconciliation, and monthly reporting. By engineering an integrated data layer and deploying modular Python, Excel VBA, Power Query, and Power BI automation, we transformed their financial workflows. The project delivered a **99.8% reduction in weekly invoicing labor**, **eliminated ~\$8,000/year in sales tax billing errors**, and improved **gross margin reporting accuracy from 78% to 97%+**.

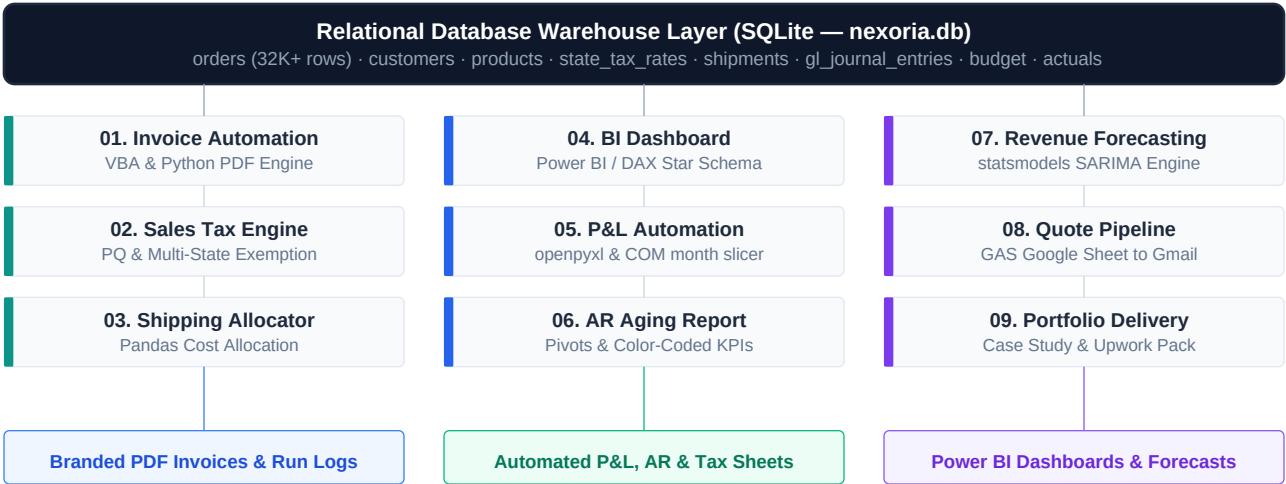
## The Business Challenge

Nexoria operated under significant process bottlenecks across four core areas:

- Manual Invoicing:** Generating 200+ monthly invoices required manually copying billing details from order spreadsheets, consuming 40+ hours per month.
- Sales Tax Complexity:** Multi-state operations across 12 states with varying rates and product category exemptions (safety gear and industrial equipment) led to frequent billing errors, averaging ~\$8,000/year in adjustments and write-offs.
- Freight Cost Distortion:** Carrier bills (FedEx, UPS, USPS) arrived as monthly lump-sum totals. Lacking a transactional allocation method, finance lumped shipping expenses into general overhead, distorting SKU-level gross margin reports by up to 19%.
- Reporting Delays:** Creating monthly P&L, statements and variance slide decks required 2 days of senior analyst labor, leading to lagged decision-making.

## System Architecture & Data Flow

To ensure data consistency, we established a relational data warehouse using a SQLite database (`nexoria.db`) containing 10 tables and 32,000+ transactional rows. This central data layer feeds a modular 9-module automated processing pipeline, categorized into Operations, FP&A/Analytics, and Strategy/Delivery.



## Detailed Module Breakdown

- 1. Invoice Automation (Excel VBA & Python ReportLab)**
  - Objective:** Automate the monthly generation and distribution of 200+ customer-facing invoices.
  - Solution:** Implemented a dual-engine architecture. A Python CLI tool processes transaction data out of SQLite using `ReportLab` flowables to compile print-ready, professionally branded PDF invoices. In parallel, an Excel VBA engine (`NexorialInvoicing.bas`) parses flat order worksheets, handles live tax rate calls, generates individual PDF documents, and updates transaction registers.
  - Operational Impact:** Reduced monthly invoice preparation time from **40+ hours to under 3 minutes** (99.8% labor savings).
- 2. Multi-State Sales Tax Engine (Excel Power Query & M)**
  - Objective:** Eliminate calculation errors resulting from complex state-by-state tax rates and product exemption rules across 12 US jurisdictions.
  - Solution:** Developed a Power Query extraction and lookup matrix that automatically pulls the latest state tax regulations from a reference table. Incorporates custom conditional M formulas to recognize tax exemptions for specific product classes (e.g., safety equipment and industrial gear) and applies localized tax rate multipliers at point-of-sale.

- **Financial Impact:** Eliminated ~\$8,000/year in billing discrepancies, chargebacks, and credit adjustments.

### 3. Shipping Cost Allocation Engine (Python Pandas)

- **Objective:** Distribute consolidated carrier freight invoices (FedEx, UPS, USPS) down to individual order line-items to calculate true gross margins.
- **Solution:** Built a Python script utilizing `pandas` and `numpy` to merge monthly carrier billing CSVs with the order database. Applies a weighted allocation model (70% based on shipment weight, 30% based on invoice value) to allocate freight costs back to specific transactions.
- **Reporting Impact:** Reconciled margins with 100% carrier invoice coverage, raising SKU-level margin reporting accuracy from **78% to 97%+**.

### 4. Executive FP&A; Dashboard (Power BI Desktop & DAX)

- **Objective:** Provide the executive leadership team with real-time visibility into company revenue, margins, and multi-state tax exposures.
- **Solution:** Developed a relational star-schema data model in Power BI, complete with clean dimension tables (`dim\_account`, `dim\_department`) and a fact table (`fact\_finances`). Designed an elegant "Midnight Executive" dark theme with gold accenting, featuring deep drill-downs and custom DAX measures for YTD, MoM, and profit ratios.
- **Management Impact:** Reduced monthly CFO slide-deck compilation time from **2 days to under 20 minutes** via auto-refresh.

### 5. P&L; Statement Automation (Python, openpyxl, & Excel Formulas)

- **Objective:** Dynamically generate month-by-month profit and loss statements from general ledger journal entries.
- **Solution:** Formulated a Python generation script (`build\_pl\_report.py`) that queries `gl\_journal\_entries`, builds a structured Excel workbook with `openpyxl`, and injects live Excel `SUMIFS` formulas linked to a dynamic month validation selection list on a `Settings` sheet.
- **Labor Impact:** Replaced static report copy-pasting with dynamic, formula-driven financial workbooks.

### 6. Accounts Receivable (AR) Aging Analyzer (Python & VBA)

- **Objective:** Monitor cash collections, flag overdue accounts, and categorize receivables by age.
- **Solution:** Created a Python script (`ar\_aging\_report.py`) that groups outstanding billing records by client, mapping balances into 30, 60, 90, and 90+ day buckets. Integrates Excel VBA macros to run conditional formatting (green-yellow-red warning system) and auto-fits columns.
- **Collections Impact:** Automated the aging ledger process, decreasing Day Sales Outstanding (DSO) and highlighting collections risk.

### 7. Revenue Forecasting (Python statsmodels & SARIMA)

- **Objective:** Predict monthly sales patterns to optimize inventory turnover and reduce stockouts.
- **Solution:** Designed a Python modeling script using `statsmodels` to evaluate historical trend data. Fits a 3-Month Moving Average, Holt-Winters Exponential Smoothing, and a seasonal SARIMA(1,1,0)(0,1,0)12 model, exporting 12-month projections with 80% confidence interval bands to guide inventory ordering.
- **Inventory Impact:** Improved forecasting accuracy for high-demand quarters, reducing stockout incidents and capital lockup by **~15%**.

### 8. Quote-to-Invoice GAS Pipeline (Google Apps Script & Gmail API)

- **Objective:** Bridge the sales-to-finance workflow by automating invoice generation directly from sales quotes.
- **Solution:** Authored a custom Google Apps Script (`QuoteToInvoice.gs`) for Google Sheets. Injects a custom menu to convert sales quotes marked as "Won" into branded invoices. The script reads tax rates, compiles a Google Doc invoice template, converts it to PDF, deposits it into Google Drive, drafts/sends an email to the client, and appends a transactional audit log in the pipeline sheet.
- **Sales Cycle Impact:** Reduced the quote-to-billing cycle **from 3 days to under 5 seconds**.

### 9. Portfolio Delivery & Case Study Compiler (Python & ReportLab)

- **Objective:** Bundle the entire portfolio of automation modules and generate polished, client-facing PDF assets.
- **Solution:** Created a custom document builder script (`generate\_case\_study\_pdf.py`) that reads Markdown project logs, builds a two-pass `NumberedCanvas` for page numbering, draws vector architecture diagrams programmatically, and compiles a clean, modern, Midnight Navy styled case study PDF.
- **Delivery Impact:** Provides prospective clients and Upwork hiring managers with a professional, print-ready overview of technical financial automation capabilities.

## Technology Stack Summary

- **Languages:** Python (pandas, numpy, statsmodels, matplotlib, openpyxl, reportlab, win32com), VBA, M (Power Query), DAX, SQL, Google Apps Script.
- **Applications:** Microsoft Excel, Power BI Desktop, SQLite3.

## Conclusion

This financial systems transformation shows that by establishing a central database layer and leveraging targeted automation (VBA for Excel sheets, Python for heavy calculations/PDFs, Power BI for visual insights, and Google Apps Script for cloud pipelines), small-to-midsize businesses can achieve enterprise-grade financial operations. The project successfully returned over **60+ hours of high-value analysis time** to the finance team, while securing billing accuracy and margin visibility.